

Lezione 2 – Tipi, riferimenti, String

Il sistema dei tipi di Java e le stringhe

Egidia Cirillo

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di Fuorigrotta (J-Z)

Agenda della lezione (2h)

- ① **Ripasso e Homework 1** (15 min)
 - uno di voi alla lavagna con Calcolatrice, Bmi o Fizzbuzz: lo commentiamo insieme
 - un minuto su stack e heap
- ② **Il sistema dei tipi** (20 min)
 - tipi primitivi, letterali, costanti; divisione intera e overflow
 - riferimenti: cosa contiene davvero una variabile; null
- ③ **Le stringhe** (20 min)
 - String: letterali, pool, == vs equals, metodi principali, immutabilità
 - concatenazione; scorrere una stringa; StringBuilder
- ④ **Esercitazione – tutti insieme** (55 min)
 - *Commentiamo insieme*: AnalisiParola
 - *Insieme, passo passo*: il cifrario di Cesare
 - un esperimento: String contro StringBuilder in un ciclo
- ⑤ **Wrap-up e Homework 2** (10 min)

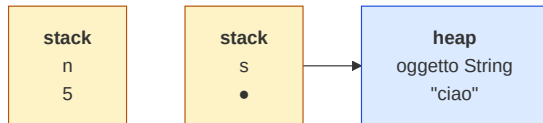
Un minuto di ripasso: stack e heap

Due zone di memoria, con regole diverse.

Stack

- **variabili locali** e **parametri** dei metodi
- spazio riservato quando il metodo *inizia*, liberato quando *termina*: automatico
- dimensione **fissa**, nota al compilatore

```
int n = 5; // solo stack
String s = new String("ciao"); // stack + heap
```

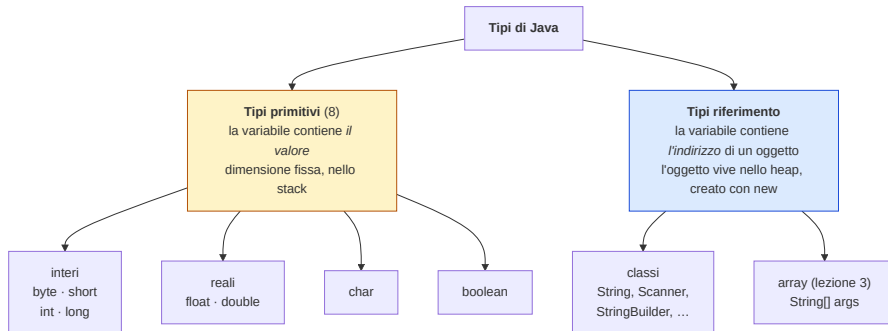


Heap

- dati creati **a tempo di esecuzione** su richiesta del programma: `malloc` in C, `new` in Java
- dimensione anche non nota in anticipo
- **sopravvivono** al metodo che li ha creati, finché qualcuno li usa

`n` è il numero 5. `s` non è la stringa: è un **riferimento**, l'indirizzo del punto dello heap dove la stringa è stata creata. Quale dei due casi vale, lo decide il **tipo**: l'argomento di oggi.

Java è fortemente tipizzato: due famiglie di tipi



Fortemente tipizzato

Ogni variabile, parametro e valore ha un tipo **noto al compilatore**, che rifiuta le operazioni incompatibili *prima* che il programma giri: `int n = "ciao";` non compila, `if (12)` nemmeno.

Gli 8 tipi primitivi

Tipo	Bit	Valori	Letterale
boolean	–	true, false	true
char	16	un carattere Unicode, da <code>'\u0000'</code> a <code>'\uFFFF'</code>	<code>'M'</code> , <code>'\n'</code>
byte	8	da -128 a 127	100
short	16	da -2^{15} a $2^{15} - 1$	30000
int	32	da -2^{31} a $2^{31} - 1$ ($\approx \pm 2,1$ miliardi)	42, 1_000_000
long	64	da -2^{63} a $2^{63} - 1$	42L
float	32	IEEE 754, ≈ 7 cifre significative	3.14f
double	64	IEEE 754, ≈ 15 cifre significative	3.14, 1e-3

- le dimensioni sono **fisse**, uguali su ogni macchina (in C dipendono dall'architettura)
- i letterali interi sono `int`, quelli reali `double`: `42L` e `3.14f` per gli altri
- apici **singoli** per i `char`, **doppi** per le `String`
- `1_000_000`: il trattino basso è solo per chi legge
- una **variabile locale** va inizializzata prima dell'uso: altrimenti *errore di compilazione*, non “valore casuale” come in C
- nel corso useremo quasi solo `int`, `double`, `char` e `boolean`

Variabili, letterali e i limiti degli interi – TipiPrimitivi.java

```
int abitanti = 913_000;
long distanzaKm = 149_600_000L;    // suffisso L
double pi = 3.14159;
float tasso = 0.035f;              // suffisso f
char iniziale = 'M';
boolean iscritto = true;

int massimo = Integer.MAX_VALUE;    // 2147483647
System.out.println(massimo + 1);    // overflow!

int a = 7, b = 2;
System.out.println(a / b);          // 3
System.out.println(a % b);          // 1
System.out.println(7.0 / b);        // 3.5

// int x;
// System.out.println(x);    // ERRORE: x non
//                          // inizializzata
```

```
-2147483648
3
1
3.5
```

Overflow. Un `int` ha 32 bit: dopo $2^{31} - 1$ si “riparte” da -2^{31} . Nessun errore, nessun avviso: il risultato è semplicemente sbagliato. Con numeri grandi si usa `long`.

Divisione. Fra due interi il risultato è **intero** (troncato, non arrotondato); `%` dà il resto. Basta che *uno* dei due operandi sia reale perché la divisione sia reale: è il motivo per cui in `Media.java` la somma era `double`.

Regola

`int / int` è la fonte numero uno di risultati “inspiegabili”. Quando vedete 3.0 dove aspettavate 3.5, cercate una divisione fra interi.

Costanti: final e la classe Math – Costanti.java

```
import static java.lang.Math.PI; // solo PI

public class Costanti {
    public static double areaCerchio(double r) {
        return PI * r * r; // e non Math.PI
    }

    public static void main(String[] args) {
        final double IVA = 0.22; // non cambia piu'
        final int MAX_TENTATIVI; // inizializzata dopo
        MAX_TENTATIVI = 3; // ...UNA sola volta
        // IVA = 0.10; // ERRORE

        double conIva = 100.0 * (1 + IVA);
        System.out.printf("%.2f%n", conIva);
        System.out.printf("%.4f%n", areaCerchio(2.0));
        System.out.println(Math.sqrt(2)); // 1.414..
        System.out.println(Math.pow(2, 10)); // 1024.0
        System.out.println(Math.round(2.5)); // 3
    }
}
```

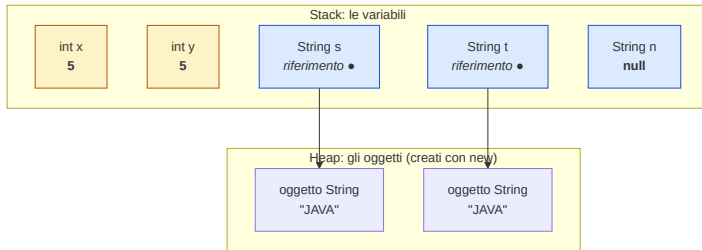
final: la variabile può ricevere **un solo** assegnamento; il compilatore blocca ogni tentativo successivo. Nome in UPPER_SNAKE_CASE.

Perché usarle: un numero “nudo” (0.22) non dice cosa rappresenta, IVA sì; se cambia, si cambia in un punto solo.

Math (in `java.lang`, sempre disponibile): costanti `PI`, `E`; metodi static `sqrt`, `pow`, `abs`, `max`, `min`, `round`, `floor`, `ceil`, `random`...

`import static` permette di scrivere `PI` al posto di `Math.PI`: comodo, ma con parsimonia (chi legge deve capire da dove arriva `PI`).

Riferimenti: cosa contiene davvero una variabile

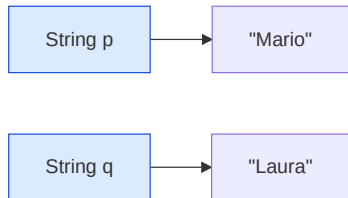


```
int x = 5;  
int y = x;           // copia il VALORE  
String s = new String("JAVA"); // crea un oggetto  
String t = new String("JAVA"); // ne crea un ALTRO  
String n = null;     // nessun oggetto
```

- una variabile **primitiva** contiene il valore
- una variabile **riferimento** contiene “l’indirizzo” di un oggetto: come un puntatore del C, ma senza aritmetica e senza `*/&`
- in Java **non esistono** variabili “di tipo oggetto”: solo riferimenti a oggetti

Le conseguenze: assegnazione, confronto, oggetti orfani

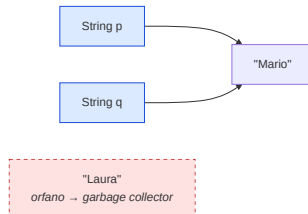
Prima di $q = p$



Assegnazione $q = p$: copia il *riferimento*, non l'oggetto. Ora p e q indicano lo **stesso** oggetto.

Confronto $p == q$: confronta i *riferimenti*: true solo se indicano lo stesso oggetto. Due oggetti con lo stesso contenuto sono $==$? **No**. Per il contenuto serve un metodo: `equals`.

Dopo $q = p$



Oggetti orfani. "Laura" non è più raggiungibile da nessuna variabile: è *garbage*. Il **garbage collector** della JVM lo libera da solo, prima o poi. Niente *free*, niente *delete*.

Parametri. Passare un oggetto a un metodo significa passare (una copia del) riferimento: il metodo lavora sullo *stesso* oggetto del chiamante. Vale per `String`, per gli array, per tutto.

null e il confronto fra riferimenti – Riferimenti.java

```
String str1 = new String("JAVA"); // oggetto 1
String str2 = new String("JAVA"); // oggetto 2
String str3 = str1;                // = oggetto 1

System.out.println(str1.equals(str2)); // true
System.out.println(str1 == str2);      // false
System.out.println(str1 == str3);      // true

String nessuno = null;
System.out.println(nessuno);           // null
System.out.println(nessuno == null);   // true
System.out.println(nessuno.length());  // BOOM
```

```
Exception in thread "main"
java.lang.NullPointerException: Cannot invoke
"String.length()" because "nessuno" is null
    at Riferimenti.main(Riferimenti.java:18)
```

null è il valore di un riferimento che **non indica alcun oggetto**. Non è 0, non è -1: è una parola riservata.

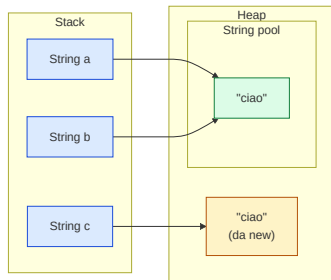
Assegnare null a una variabile “lascia andare” l'oggetto che indicava, che può diventare garbage.

String: letterali, oggetti e il *pool*

Una stringa è una **sequenza di caratteri**: un **oggetto** della classe `String` (`java.lang`).

```
String a = "ciao";           // letterale
String b = "ciao";           // stesso di a!
String c = new String("ciao"); // oggetto nuovo
System.out.println(a == b);   // true
System.out.println(a == c);   // false
System.out.println(a.equals(c)); // true
```

- i **letterali** uguali sono *un solo oggetto*, conservato nello **string pool**; dividerlo è sicuro perché `String` è **immutabile** (fra poco)
- `new String(...)` crea sempre un oggetto *nuovo*: quasi mai necessario, **usate i letterali**



E per i tipi primitivi?

Nessun pool, e non serve: con `int a = 1, b = 1`; ogni variabile contiene la *propria copia* del valore, e `a == b` è true perché confronta direttamente i **valori**. Il pool riguarda solo i letterali `String`.

Confrontare stringhe: equals, equalsIgnoreCase, compareTo

```
String u = "diego";  
u.equals("Diego")           // false  
u.equalsIgnoreCase("Diego") // true  
  
"abc".compareTo("abd")      // -1: abc < abd  
"b".compareTo("a")          // 1: b > a  
"abc".compareTo("abc")      // 0: uguali  
"Zoe".compareTo("anna")     // -7: 'Z'=90 < 'a'=97
```

Il confronto è **lessicografico**: carattere per carattere, secondo il *codice* Unicode di ciascuno. Le maiuscole ('A'=65...) vengono *prima* delle minuscole ('a'=97...): per un ordine “da dizionario” si confrontano le versioni `toLowerCase()`.

L'errore classico

```
String risposta = in.next();  
if (risposta == "si") { ... } // NO!  
if (risposta.equals("si")) { ... }
```

La stringa letta da tastiera è un oggetto *nuovo*: `==` con il letterale "si" è false anche se l'utente ha scritto proprio si. A volte `==` “funziona” per caso (grazie al pool): non fidatevi.

`compareTo` restituisce un `int`: `< 0`, `0` o `> 0`.
Serve per *ordinare*: lo ritroveremo nella lezione 12 con `Comparable`.

I metodi principali di String – MetodiString.java

```
String s = " Diego Armando Maradona ";
String t = s.trim(); // senza spazi ai bordi
t.length()           // 22
t.charAt(0)           // 'D'
t.charAt(t.length() - 1) // 'a'
t.indexOf("Arm")       // 6
t.indexOf('a')         // 9 (la prima 'a')
t.indexOf("Pele")      // -1 (non c'è)
t.substring(6, 13)     // "Armando" (13 escluso)
t.substring(14)        // "Maradona"
t.toUpperCase()        // "DIEGO ARMANDO MARADONA"
t.replace('a', '4')    // "Diego Arm4ndo M4r4don4"
t.contains("go")       // true
t.startsWith("Di")    // true
```

Domanda	Metodo
quanto è lunga?	length()
il carattere in posizione <i>i</i> ?	charAt(<i>i</i>)
dove compare...?	indexOf(<i>x</i>), indexOf(<i>x</i> , <i>da</i>)
contiene...?	contains, startsWith, endsWith
una parte	substring(<i>i</i> , <i>j</i>), substring(<i>i</i>)
una copia modificata	toUpperCase, toLowerCase, trim, replace, concat
è uguale a...?	equals, equalsIgnoreCase
viene prima di...?	compareTo

- gli indici partono da **0**; l'ultimo è `length() - 1`; `charAt(22)` su 22 caratteri ferma il programma (`StringIndexOutOfBoundsException`)
- `indexOf` restituisce `-1` se non trova nulla: **controllatelo**
- l'elenco completo: documentazione *Java SE 25 API*, classe `String`

Le stringhe sono immutabili: il risultato “perso” – Immutabile.java

```
String nome = "diego";

nome.toUpperCase();           // calcolato e perso
System.out.println(nome);    // diego

String m = nome.toUpperCase(); // salvato
System.out.println(m);       // DIEGO
System.out.println(nome);    // diego (intatto)

nome = nome.toUpperCase();    // riassegno
System.out.println(nome);    // DIEGO

String saluto = "ciao";
saluto.concat(" a tutti");   // perso
System.out.println(saluto);  // ciao
```

Immutabile

Una volta creato, un oggetto String **non cambia mai**. Nessun metodo modifica la stringa su cui è chiamato: toUpperCase, trim, replace, substring, concat... **restituiscono una nuova stringa**.

Quindi il valore restituito va *sempre* usato: assegnato a una variabile, stampato, passato a un metodo. Una chiamata “da sola” su una riga è quasi certamente un errore.

Perché è fatta così? Il pool funziona solo perché nessuno può modificare l'oggetto condiviso; una stringa passata a un metodo torna *intatta*; e come per i numeri, $3 + 4$ non cambia il 3.

Concatenazione con +: da tutto a stringa – Concatenazione.java

```
String nome = "Anna";   int esami = 7;
String riga = nome + " ha " + esami + " esami";
// "Anna ha 7 esami"

System.out.println("" + 1 + 2);           // 12
System.out.println(1 + 2 + "");           // 3
System.out.println("S: " + (1 + 2));       // S: 3

String s = "a";
s += "b";                                 // "ab"
s += 3;                                   // "ab3"

String daNumero = String.valueOf(42);     // "42"
int daStringa = Integer.parseInt("42");
System.out.println(daNumero + 1);          // 421
System.out.println(daStringa + 1);         // 43
```

La regola. Se *almeno uno* dei due operandi di + è una String, l'altro viene convertito in stringa e i due vengono concatenati. Altrimenti + è la somma.

L'ordine conta. + si valuta da sinistra a destra: in "" + 1 + 2 il primo + è già una concatenazione; in 1 + 2 + "" il primo è una somma. Le parentesi risolvono ogni dubbio.

Conversioni esplicite. String.valueOf(x) da qualunque valore a stringa;
Integer.parseInt(s) e
Double.parseDouble(s) da stringa a numero (è ciò che Scanner.nextInt() fa per voi).

s += x crea una **nuova** stringa e la riassegna a s: comodo, ma dentro un ciclo lungo costa caro. Lo misureremo.

Scorrere una stringa carattere per carattere – ScorriStringa.java

```
/** Quante vocali contiene s? */
public static int contaVocali(String s) {
    int vocali = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if ("aeiouAEIOU".indexOf(c) >= 0) {
            vocali++;
        }
    }
    return vocali;
}

/** Quante volte c compare in s? */
public static int conta(String s, char c) {
    int n = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == c) { // fra char: ok
            n++;
        }
    }
    return n;
}
```

Il **pattern** che userete decine di volte:

- 1 for da 0 a s.length() - 1
- 2 char c = s.charAt(i)
- 3 una decisione su c

Due trucchi

- "aeiou".indexOf(c) >= 0 chiede "c è una delle vocali?" senza cinque confronti
- char è un tipo **primitivo**: fra char il confronto == è corretto (è fra String che non va bene)

```
vocali in "Programmare in Java": 7
'a' compare 4 volte
' ' compare 2 volte
```

StringBuilder: quando la stringa deve cambiare – UsaStringBuilder.java

```
StringBuilder sb = new StringBuilder("Diego");
sb.length()           // 5
sb.capacity()          // 21 = 5 + 16
sb.append(" 10");       // "Diego 10"
sb.insert(0, "Don ");   // "Don Diego 10"
sb.setCharAt(0, 'd');   // "don Diego 10"
sb.deleteCharAt(11);    // "don Diego 1"
sb.reverse();           // "1 ogeiD nod"
String r = sb.toString(); // torna una String

StringBuilder alfabeto = new StringBuilder();
for (char c = 'a'; c <= 'z'; c++) {
    alfabeto.append(c);
}
System.out.println(alfabeto); // abc...xyz
```

Una sequenza di caratteri **modificabile**: i metodi cambiano l'oggetto *in place* e restituiscono l'oggetto stesso (si possono concatenare: `sb.append(x).reverse()`).

Capacità: spazio riservato (lunghezza iniziale +16); quando finisce, cresce da sola. Se sapete quanto sarà lunga: `new StringBuilder(n)`.

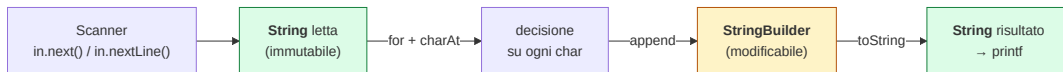
Metodi: `append` (per ogni tipo), `insert`, `delete`, `deleteCharAt`, `setCharAt`, `reverse`, `replace`; `length`, `charAt` come in `String`; `toString` per ottenere la `String` finale.

Quando usarlo

Quando **costruite** una stringa pezzo per pezzo, di solito in un ciclo. Per tutto il resto, `String`.

Obiettivo dell'esercitazione

Tre programmi, tutti insieme, tutti sulla stessa idea: **leggere una stringa, scorrerla, costruirne un'altra.**



- 1 **Commentiamo insieme** (15') – `AnalisiParola.java`: lo leggiamo riga per riga; contiene tutto ciò che serve dopo.
- 2 **Insieme, passo passo** (30') – `Cesare.java`: io al PC proiettato, voi in VS Code. Prima decidiamo il disegno, poi scriviamo.
- 3 **Un esperimento** (10') – `ConfrontoConcatenazione.java`: quanto costa davvero `s += x` in un ciclo?

Commentiamo insieme – AnalisiParola.java

```
import java.util.Scanner;

public class AnalisiParola {

    public static int contaVocali(String s) {
        int n = 0;
        for (int i = 0; i < s.length(); i++) {
            if ("aeiou".indexOf(s.charAt(i)) >= 0) { n++; }
        }
        return n;
    }

    public static String rovescia(String s) {
        StringBuilder sb = new StringBuilder(s);
        return sb.reverse().toString();
    }

    public static boolean isPalindroma(String s) {
        return s.equals(rovescia(s)); // equals, non ==
    }

    public static void main(String[] args) { /* prossima slide */ }
}
```

Commentiamo insieme – il main

```
public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Scrivi una parola: ");
    String parola = in.next().toLowerCase();    // una parola, tutta minuscola
    in.close();

    int lunghezza = parola.length();
    char prima = parola.charAt(0);
    char ultima = parola.charAt(lunghezza - 1);

    System.out.printf("Parola:      %s\n", parola);
    System.out.printf("Lunghezza:   %d\n", lunghezza);
    System.out.printf("Maiuscolo:  %s\n", parola.toUpperCase());
    System.out.printf("Iniziale:   %c   Finale: %c\n", prima, ultima);
    System.out.printf("Vocali:     %d   Consonanti: %d\n",
        contaVocali(parola), lunghezza - contaVocali(parola));
    System.out.printf("Rovesciata: %s\n", rovescia(parola));
    System.out.printf("Palindroma? %b\n", isPalindroma(parola));
    System.out.printf("Stessi estremi? %b\n", prima == ultima);
}
```

Tre variabili locali (lunghezza, prima, ultima) calcolate una volta e riusate; poi una riga di output per ogni domanda sulla parola. Ogni printf usa lo specificatore giusto per il **tipo**: %s per String, %d per int, %c per char, %b per boolean.

Commentiamo insieme – eseguiamo e discutiamo

```
$ javac AnalisiParola.java
$ java AnalisiParola
Scrivi una parola: Anna
Parola:      anna
Lunghezza:   4
Maiuscolo:   ANNA
Iniziale:    a   Finale: a
Vocali:      2   Consonanti: 2
Rovesciata:  anna
Palindroma?  true
Stessi estremi? true
```

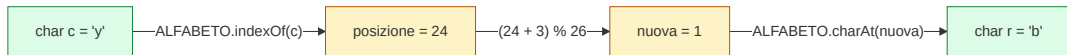
Poi proviamo: cosa succede con Anna Maria?
Con JAVA? E premendo Invio senza scrivere nulla?

Leggiamolo dall'alto, e rispondiamo:

- 1 `in.next().toLowerCase()`: due chiamate in fila. Quale oggetto riceve `toLowerCase`? Cosa contiene parola?
- 2 perché l'ultimo carattere è in `lunghezza - 1` e non in `lunghezza`?
- 3 `prima == ultima`: qui `==` è corretto. Perché? (Che tipo hanno `prima` e `ultima`?)
- 4 in `isPalindroma`, cosa cambierebbe con `==` al posto di `equals`?
- 5 `rovescia` crea uno `StringBuilder` e lo rovescia: la parola originale è cambiata?
- 6 `contaVocali` è chiamato *due volte* con lo stesso argomento: come lo evitereste?

Insieme, passo passo (30 min) – il cifrario di Cesare

Il problema. Leggere un testo e una chiave k ($0 \leq k \leq 25$); stampare il testo cifrato sostituendo ogni lettera con quella k posizioni più avanti nell'alfabeto (dopo la z si ricomincia dalla a); gli altri caratteri restano invariati. Poi decifrarlo e verificare che torni uguale.



Prima di scrivere codice, decidiamo insieme:

- 1 come “spostare” una lettera senza fare conti sui caratteri?
→ una costante `ALFABETO = "abc...z"`: `indexOf` dà la posizione, `charAt` la lettera nella nuova posizione
- 2 come tornare a capo dopo la z? → l'operatore %
- 3 quali **metodi**: `sposta(char, int)`, `cifra(String, int)`, `decifra(String, int)`
- 4 cosa fa il `main`: legge, chiama, stampa

Come procediamo

Una riga alla volta: io al PC proiettato, voi sul vostro portatile in VS Code. Prima sposta (e la proviamo subito su una lettera), poi cifra, poi il `main`. Compiliamo ed eseguiamo **da terminale** almeno una volta.

Domanda da tenere a mente: decifrare è un'operazione *nuova* o è ancora cifrare, con un'altra chiave?

Una possibile soluzione – Cesare.java (1/2)

```
public class Cesare {

    public static final String ALFABETO = "abcdefghijklmnopqrstuvwxyz";

    public static char sposta(char c, int chiave) {
        int posizione = ALFABETO.indexOf(c);
        if (posizione < 0) {                // non e' una lettera minuscola
            return c;                      // resta com'e'
        }
        int nuova = (posizione + chiave) % 26;    // "torna a capo" dopo la z
        return ALFABETO.charAt(nuova);
    }

    public static String cifra(String testo, int chiave) {
        StringBuilder risultato = new StringBuilder();
        for (int i = 0; i < testo.length(); i++) {
            risultato.append(sposta(testo.charAt(i), chiave));
        }
        return risultato.toString();
    }
}
```

static final: una costante di classe, condivisa da tutti i metodi. sposta fa *una cosa sola* ed è facile da provare da sola: sposta('y', 3) deve dare 'b', sposta('!', 3) deve dare '!'. cifra è il pattern di oggi: for + charAt + append + toString; non tocca testo, non potrebbe nemmeno volendo.

Una possibile soluzione – Cesare.java (2/2)

```
// Decifrare = cifrare con la chiave complementare
public static String decifra(String testo, int chiave) {
    return cifra(testo, 26 - chiave % 26);
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Testo da cifrare: ");
    String testo = in.nextLine().toLowerCase();
    System.out.print("Chiave (0-25): ");
    int chiave = in.nextInt();
    in.close();

    String cifrato = cifra(testo, chiave);
    String decifrato = decifra(cifrato, chiave);
    boolean uguale = decifrato.equals(testo);
    System.out.println("Cifrato: " + cifrato);
    System.out.println("Decifrato: " + decifrato);
    System.out.println("Uguale? " + uguale);
}
```

```
Testo da cifrare: attaccare all'alba!
Chiave (0-25):      3
Cifrato:   dwdfdduh doo'doed!
Decifrato: attaccare all'alba!
Uguale? true
```

Decifrare non è un'operazione nuova:
spostare indietro di k equivale a spostare
avanti di $26 - k$: una riga che *riusa* cifra.

`nextLine()` e non `next()`: il testo
contiene spazi. `toLowerCase()` subito,
così sposta conosce solo le minuscole.

Se avanza tempo: conservare le
maiuscole; chiave negativa o > 25 : cosa fa
%?; *forza bruta*: le 26 decifrate possibili
di un testo con chiave ignota.

Un esperimento – String contro StringBuilder in un ciclo

```
public static String conPiu(int n) {
    String s = "";
    for (int i = 0; i < n; i++) {
        s += i % 10;    // NUOVA stringa ogni volta
    }
    return s;
}
```

```
public static String conStringBuilder(int n) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < n; i++) {
        sb.append(i % 10);    // modifica in place
    }
    return sb.toString();
}
```

```
long t0 = System.nanoTime();
String a = conPiu(n);
long t1 = System.nanoTime();
String b = conStringBuilder(n);
long t2 = System.nanoTime();
// (t1 - t0) / 1e6 = millisecondi di conPiu
```

Sulla macchina del docente (java
ConfrontoConcatenazione 100000):

```
n = 100000 caratteri
String +=      :    1077.9 ms
StringBuilder :      5.2 ms
stesso contenuto? true
```

Perché 200 volte tanto? Al giro i , $s += x$ crea una nuova stringa e vi *copia* gli i caratteri di s : in tutto $1 + 2 + \dots + n \approx n^2/2$ copie.

StringBuilder scrive nel proprio spazio: $\approx n$ operazioni.

Provate voi con $n = 10^4, 10^5, 10^6$: come cresce ciascun tempo quando n cresce di 10 volte?

Ma + non è vietato: per unire *poche* stringhe ("Ciao " + nome) è perfetto. Il problema è **dentro un ciclo**.

Riepilogo: cosa abbiamo trattato

Concetto (unità di riferimento)	Dove compare oggi
Tipi primitivi, letterali, dimensioni fisse (U8)	tabella degli 8 tipi; <code>TipiPrimitivi</code>
Divisione intera, %, overflow (U8)	<code>TipiPrimitivi</code> ; % in Cesare
Costanti <code>final</code> , <code>Math</code> , <code>import static</code> (U8)	Costanti; <code>ALFABETO</code> in Cesare
Riferimenti, stack/heap, assegnazione, orfani, GC (U8)	diagrammi di memoria; <code>Riferimenti</code>
<code>null</code> , <code>NullPointerException</code> (U8)	<code>Riferimenti</code>
Letterali vs <code>new</code> , string pool (U12)	diagramma del pool
<code>==</code> vs <code>equals</code> , <code>compareTo</code> (U8, U12)	<code>Riferimenti</code> ; <code>isPalindroma</code> ; “Torna uguale?”
Metodi di <code>String</code> , indici da 0 (U12)	<code>MetodiString</code> ; <code>AnalisiParola</code> ; Cesare
Immutabilità, il risultato “perso” (U12)	Immutabile; rovescia non tocca la parola
Concatenazione, <code>valueOf</code> , <code>parseInt</code> (U12)	Concatenazione
<code>StringBuilder</code> e il suo perché (U12)	<code>UsaStringBuilder</code> ; cifra; l'esperimento

Prossima lezione (L3): metodi – passaggio per valore, riferimenti come parametri, overloading – e **array** mono e bidimensionali: finalmente potremo tenere insieme *molti* valori.

Homework 2 (da svolgere entro la prossima lezione)

- 1 ContaCaratteri.java: legge una frase (`nextLine`) e stampa quante vocali, consonanti, cifre, spazi e altri caratteri contiene, più il numero (approssimato) di parole. Un metodo static boolean per ogni categoria: `isVocale(char)`, `isLettera(char)`, `isCifra(char)`.
Suggerimento: `c >= 'a' && c <= 'z'`.
- 2 Palindromo.java: legge una frase e dice se è palindroma *ignorando* spazi, maiuscole e punteggiatura (*"I topi non avevano nipoti"* lo è). Un metodo `soloLettere(String)` che costruisce con `StringBuilder` la versione "pulita", poi la logica di `AnalisiParola`.
- 3 Acronimo.java: legge una frase e stampa le iniziali delle parole in maiuscolo: *"java virtual machine"* → JVM. Deve funzionare anche con spazi doppi o ai bordi (`trim`).

Usate solo ciò che abbiamo visto finora: niente array, niente `Character`. Se vi sembra di averne bisogno, probabilmente state complicando il problema.

Domande?

`egidia.cirillo@unina.it`